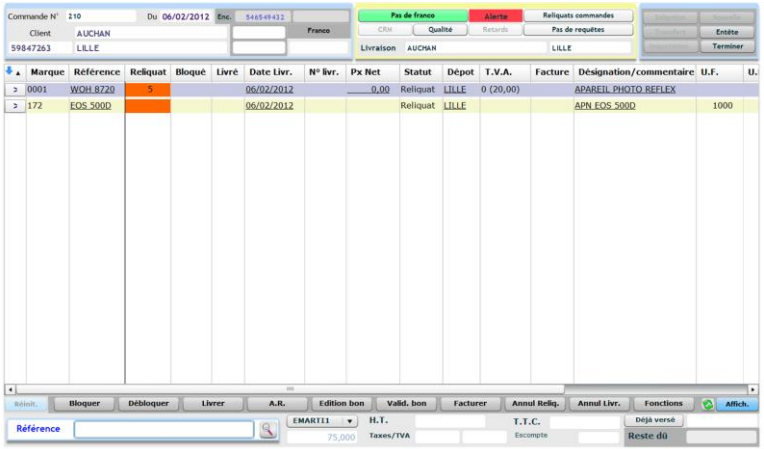


INTRODUCTION A L'ERP

POLYEDRE INFORMATIQUE

Julien DESQUENNE

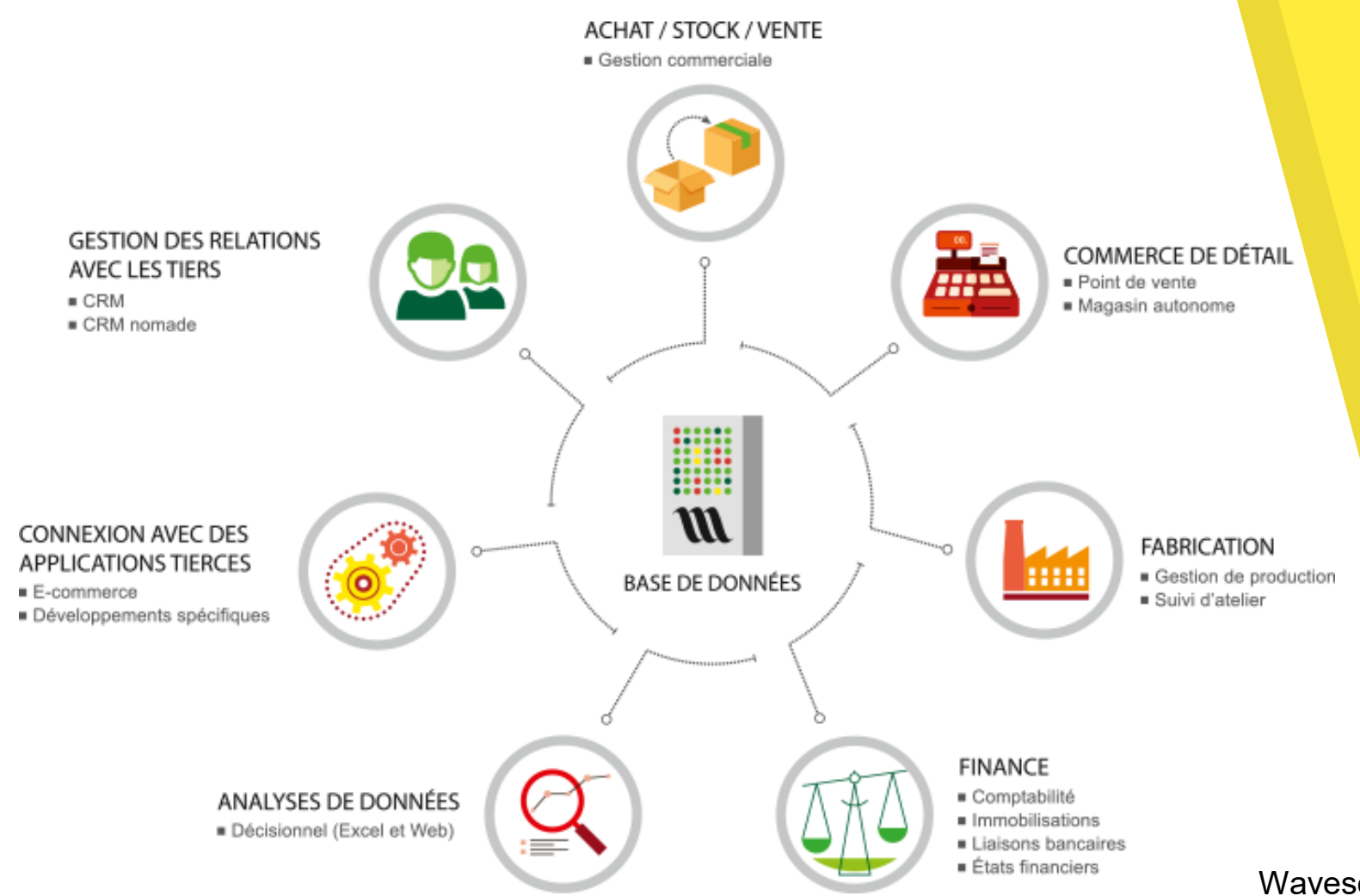
Rappel : Définition de l'ERP

ERP = Entreprise Resource Planning
littéralement : "Planification des ressources de l'entreprise"

traduit en français par :

PGI = Progiciel de gestion intégré

(Logiciel = une application / progiciels = composé de plusieurs logiciels)


ACHAT / STOCK / VENTE
Gestion commerciale
GESTION DES RELATIONS AVEC LES TIERS
CRM
CRM nomade
COMMERCE DE DÉTAIL
Point de vente
Magasin autonome
CONNEXION AVEC DES APPLICATIONS TIERCES
E-commerce
Développements spécifiques
BASE DE DONNÉES
FABRICATION
Gestion de production
Suivi d'atelier
ANALYSES DE DONNÉES
Décisionnel (Excel et Web)
FINANCE
Comptabilité
Immobilisations
Liaisons bancaires
États financiers

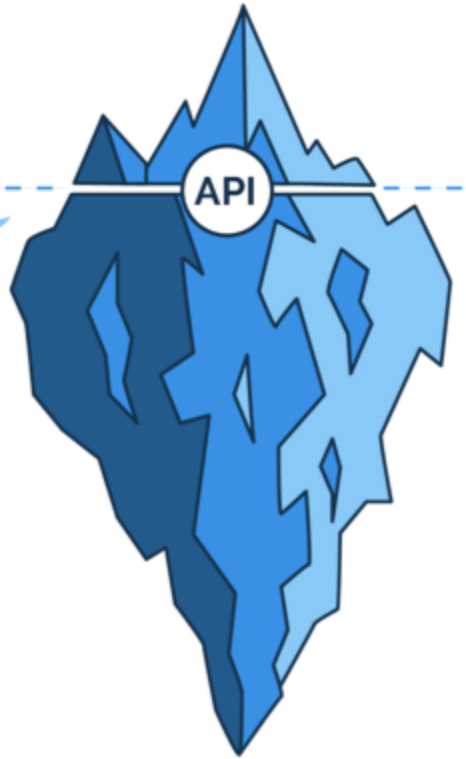
Wavesoft

Et techniquement ?

Frontend

Users see

20% of total effort



Backend

Users don't see

80% of total effort

Repetitive

Le **frontend** correspond à tout ce que l'utilisateur **voit** et **avec quoi il interagit**.

C'est l'interface graphique de l'application.

Rôles du frontend :
✅ Afficher les informations à l'utilisateur
✅ Récupérer des données depuis le backend et les afficher
✅ Permettre à l'utilisateur d'interagir avec l'application (clics, formulaires, navigation...)

📌 **Technologies utilisées :**
- 🌍 **Sur le web** : HTML, CSS, JavaScript (avec Angular, React, Vue.js, etc.)
- 📱 **Sur mobile** : Flutter (Dart), React Native, Swift (iOS), Kotlin (Android)
- 🖥️ **Sur desktop** : Flutter, Electron.js, .NET, etc.

Le **backend** est la partie cachée qui gère la **logique** et les **données** de l'application. Il tourne sur un serveur et répond aux demandes du frontend.

📌 **Rôles du backend :**
✅ Stocker et gérer les **données** (via une base de données)
✅ Réaliser des calculs ou de la logique métier
✅ Fournir des **API** pour permettre au frontend de récupérer les données
✅ Gérer l'authentification des utilisateurs

📌 **Technologies utilisées :**
- 🖥️ **Langages backend** : Python (Django, Flask), Java (Spring Boot), JavaScript (Node.js), PHP, C#, COBOL, etc.
- 🗄️ **Bases de données** : MySQL, PostgreSQL, DB2, MongoDB, Firebase, etc.
- 🌐 **APIs** : ex : REST (JSON)

Une **API** (**Interface de Programmation d'Application**) est un **pont** qui permet à différentes applications de **communiquer entre elles**.

📌 **Exemple simple** :

l’API est comme un **serveur dans un restaurant** :

- Le client (le frontend) passe la commande.
- Le serveur (API) la transmet en cuisine (backend).
- La cuisine prépare le plat et le serveur te le ramène.

L’API fonctionne de la même manière : elle transmet des **demandes** et **réponses** entre un client (frontend) et un serveur (backend).

Architecture des ERP

ERP Monolithique (ancien modèle)

- Backend et frontend sont intégrés dans une seule application.

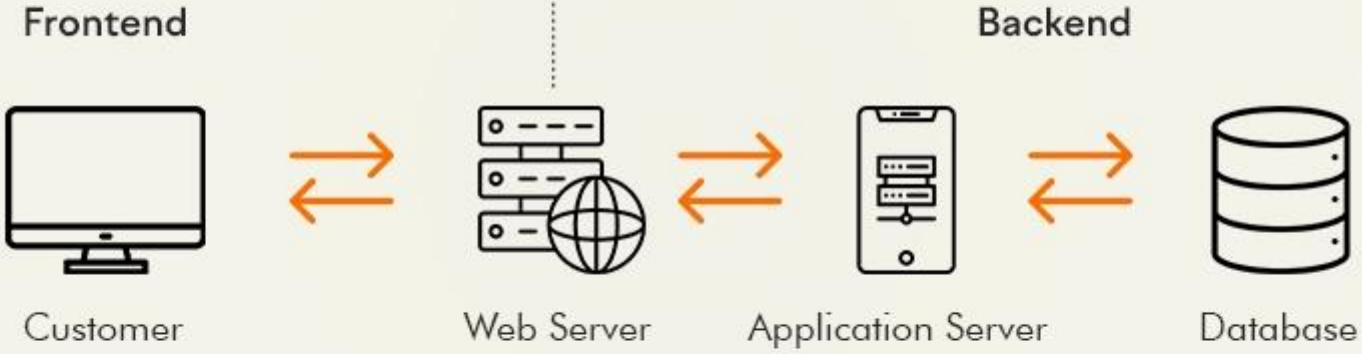
  Interface souvent desktop (Windows, macOS)
  Backend et frontend dans le même exécutable
  => difficile à faire évoluer

ERP Client / Serveur (local ou cloud)

- Backend et frontend totalement séparés.

  Frontend : installé sur le poste client OU site web / appli mobile
  Backend : base de données et logique métier sur un serveur
  Communication via API

Frontend vs Backend



Exemple de webservice qu’on va utiliser en TP

getart.cgi : Retourne les articles pour un n° de fournisseur passé en entrée

http://polyedre.eu:8000/polyfx/cgi/getart.cgi?frs=401000394


```
[
  {
    "id": 260311000000003,
    "name": "DESQUENNE A 5 PRO",
    "price": 180,
    "image": "https://m.media-amazon.com/images/I/71XS4zsY3CL._AC_SL1500_.jpg",
    "description": "A5 Pro 5G "
  },
  {
    "id": 260311000000004,
    "name": "DESQUENNEA 6 PRO",
    "price": 219,
    "image": "https://m.media-amazon.com/images/I/61A0e-Bl46L._AC_SL1500_.jpg",
    "description": "A6 Pro 5G Smartphone Débloqué "
  },
  {
    "id": 260122000000046,
    "name": "DESQUENNEFINDX 9",
    "price": 500,
    "image": "https://m.media-amazon.com/images/I/61fctUjwT9L._AC_SL1500_.jpg",
    "description": "OPPO Find X9 5G 12Go 512Go Noir Écran 6. "
  },
  {
    "id": 260311000000002,
    "name": "DESQUENNERENO 14 F",
    "price": 319,
    "image": "https://m.media-amazon.com/images/I/41x0C2wTQCL._AC_.jpg",
    "description": "Reno 14F 5G "
  },
  {
    "id": 260311000000001,
    "name": "DESQUENNERENO 15 PRO",
    "price": 709,
    "image": "https://m.media-amazon.com/images/I/61b4ErkenPL._AC_SL1500_.jpg",
    "description": "Reno 15 Pro 5G "
  }
]
```

Backend : présentation d’un code

Frontend : utilisation de Flutter

Flutter : tout est un widget

Un **widget** est **l'élément de base** de toute application Flutter. Tout ce que vous voyez à l'écran (boutons, textes, images, mises en page) est un **widget**.

📌 **Flutter fonctionne comme une boîte de Lego** :
Chaque **petit élément** est un **widget**.
On les **assemble** pour construire des interfaces complexes.

Frontend : utilisation de Flutter

Import des widget material

Point d'entrée de l'application

Démarre l'application

Widget pour centrer

Widget pour afficher le texte « Hello World »

main.dart

lib > main.dart > main

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(
    Center(
      child: Text(
        'Hello, world!',
        textDirection: TextDirection.ltr,
      ), // Text
    ), // Center
  );
}
```

Frontend : utilisation de Flutter

Deux types de widgets :

1- StatelessWidget (widget statique)
-> Ne change pas après son affichage.
Exemples : un texte, une icône, un bouton simple.

Exemple d'un StatelessWidget (texte affiché à l'écran)

```
import 'package:flutter/material.dart';

class MonWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Text("Bonjour !");
  }
}
```

2- StatefulWidget (widget dynamique) -> Peut changer en fonction des interactions de l'utilisateur.

Exemples : un compteur, un champ de texte, un bouton qui change de couleur.

```
class MonWidget extends StatefulWidget {
  @override
  _MonWidgetState createState() => _MonWidgetState();
}

class _MonWidgetState extends State<MonWidget> {
  int _compteur = 0;

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: () => setState(() => _compteur++),
      child: Center(child: Text('$_compteur', style: TextStyle(fontSize: 30))),
    );
  }
}
```



Installation
Exercices

Objectifs

1) Chaque étudiant installe les outils nécessaires

2) Création guidée d’une application flutter

3) Utilisation des webservices de l’ERP

Outils nécessaires

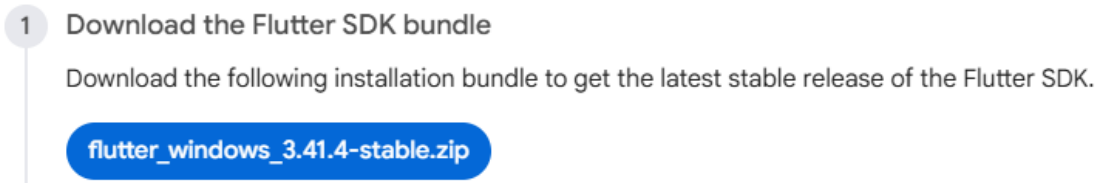
1) Git : https://git-scm.com/downloads

2) VS Code : https://code.visualstudio.com/

3) Flutter

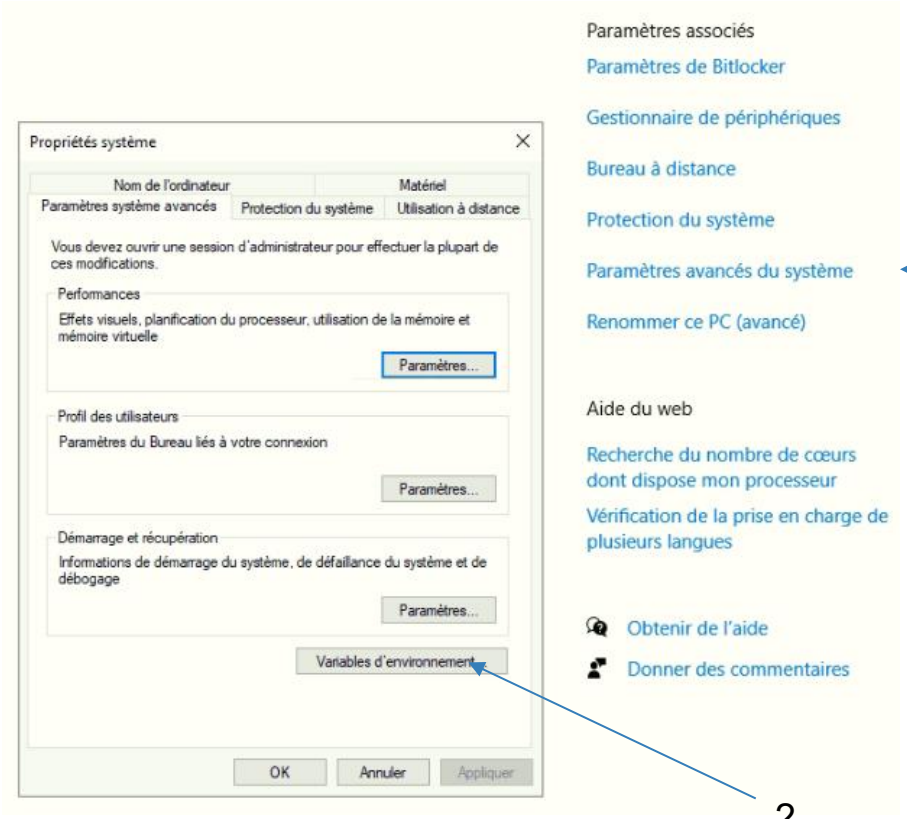
En résumé pour Windows :

-télécharger l'archive sur : https://docs.flutter.dev/install/manual

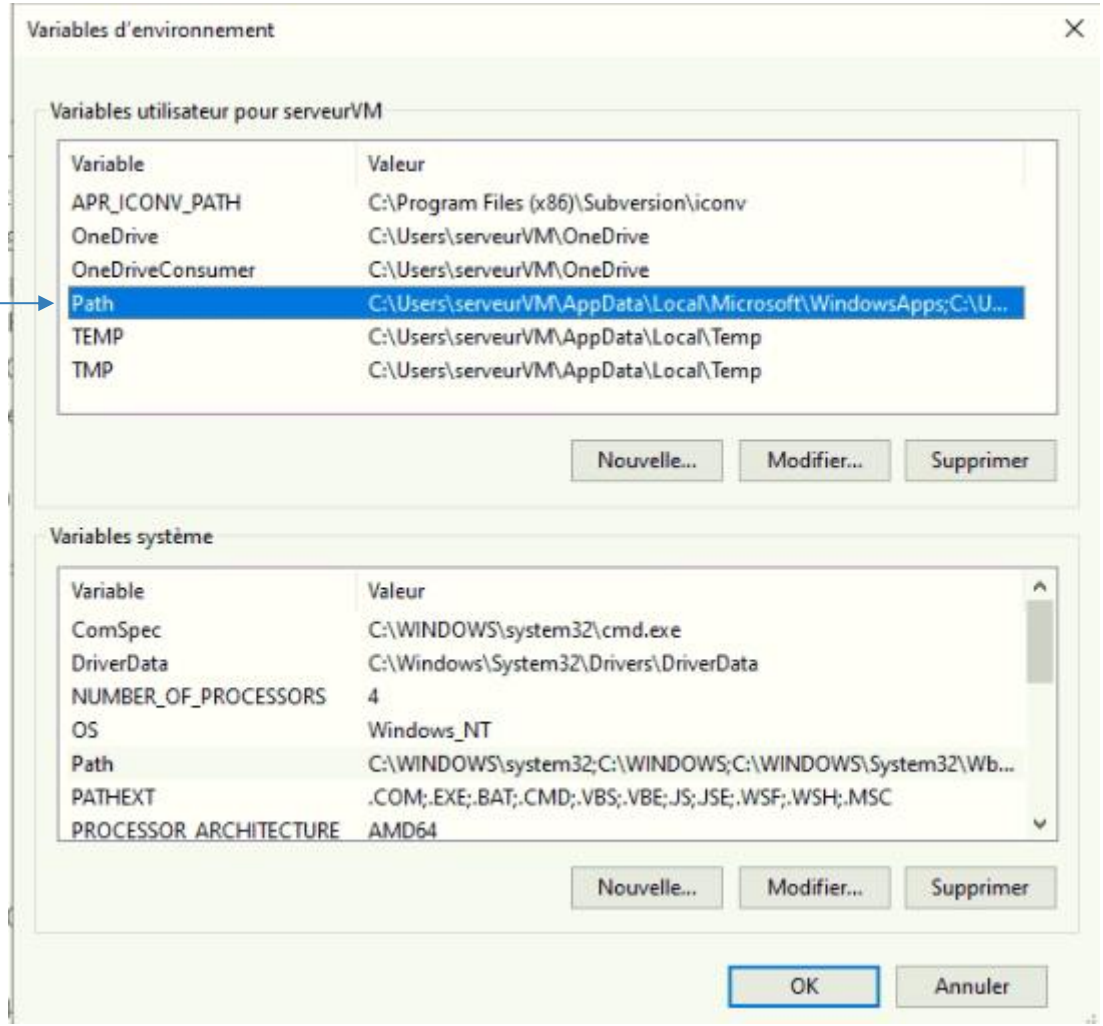


- le dézipper dans C:/dev/ par exemple

- Appuyez sur Windows + Pause (ou Windows + Fn + B)


Paramètres associés
Paramètres de Bitlocker
Gestionnaire de périphériques
Bureau à distance
Protection du système
Paramètres avancés du système
Renommer ce PC (avancé)
Aide du web
Recherche du nombre de cœurs dont dispose mon processeur
Vérification de la prise en charge de plusieurs langues
Obtenir de l'aide
Donner des commentaires
Propriétés système
Nom de l'ordinateur
Matériel
Paramètres système avancés
Protection du système
Utilisation à distance
Vous devez ouvrir une session d'administrateur pour effectuer la plupart de ces modifications.
Performances
Effets visuels, planification du processeur, utilisation de la mémoire et mémoire virtuelle
Paramètres...
Profil des utilisateurs
Paramètres du Bureau liés à votre connexion
Paramètres...
Démarrage et récupération
Informations de démarrage du système, de défaillance du système et de débogage
Paramètres...
Variables d'environnement...
OK
Annuler
Appliquer
1
2

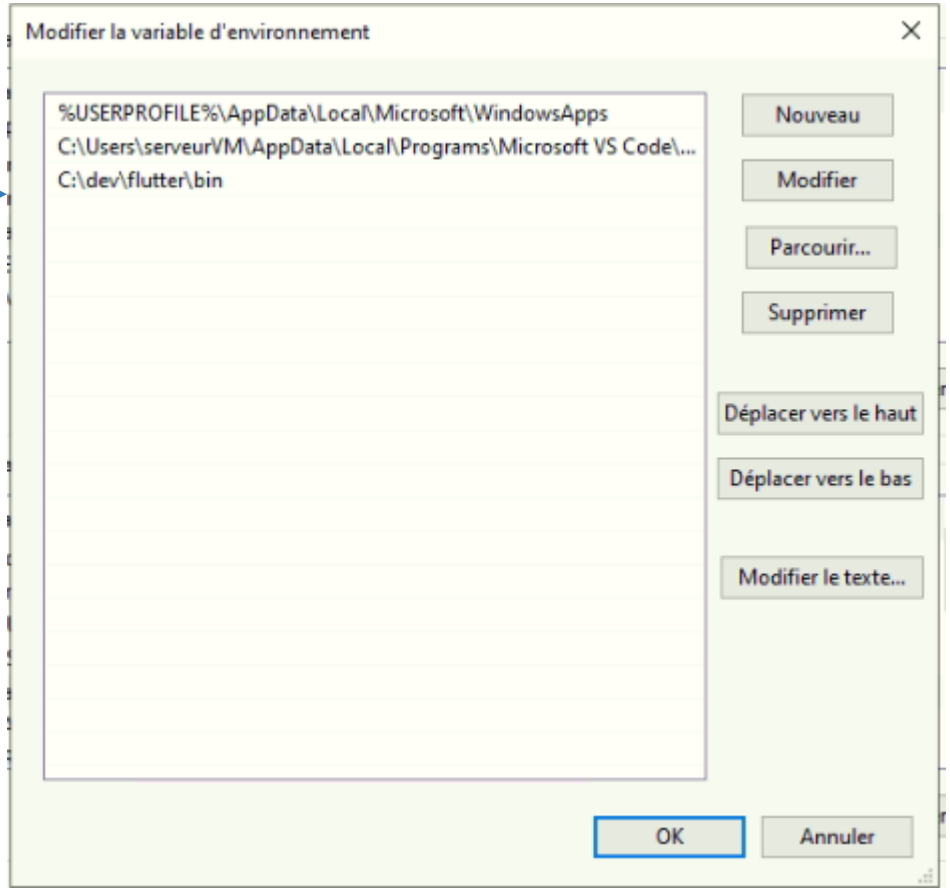
Double clic


Variables d'environnement
Variables utilisateur pour serveurVM
Variable
Valeur
APR_ICONV_PATH
C:\Program Files (x86)\Subversion\iconv
OneDrive
C:\Users\serveurVM\OneDrive
OneDriveConsumer
C:\Users\serveurVM\OneDrive
Path
C:\Users\serveurVM\AppData\Local\Microsoft\WindowsApps;C:\U...
TEMP
C:\Users\serveurVM\AppData\Local\Temp
TMP
C:\Users\serveurVM\AppData\Local\Temp
Nouvelle...
Modifier...
Supprimer
Variables système
Variable
Valeur
ComSpec
C:\WINDOWS\system32\cmd.exe
DriverData
C:\Windows\System32\Drivers\DriverData
NUMBER_OF_PROCESSORS
4
OS
Windows_NT
Path
C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\Wb...
PATHEXT
.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE
AMD64
Nouvelle...
Modifier...
Supprimer
OK
Annuler

Double clic sur une ligne vide et ajouter le chemin où vous avez extrait flutter

Ex:
C:\dev\flutter\bin

Faire OK 3 fois



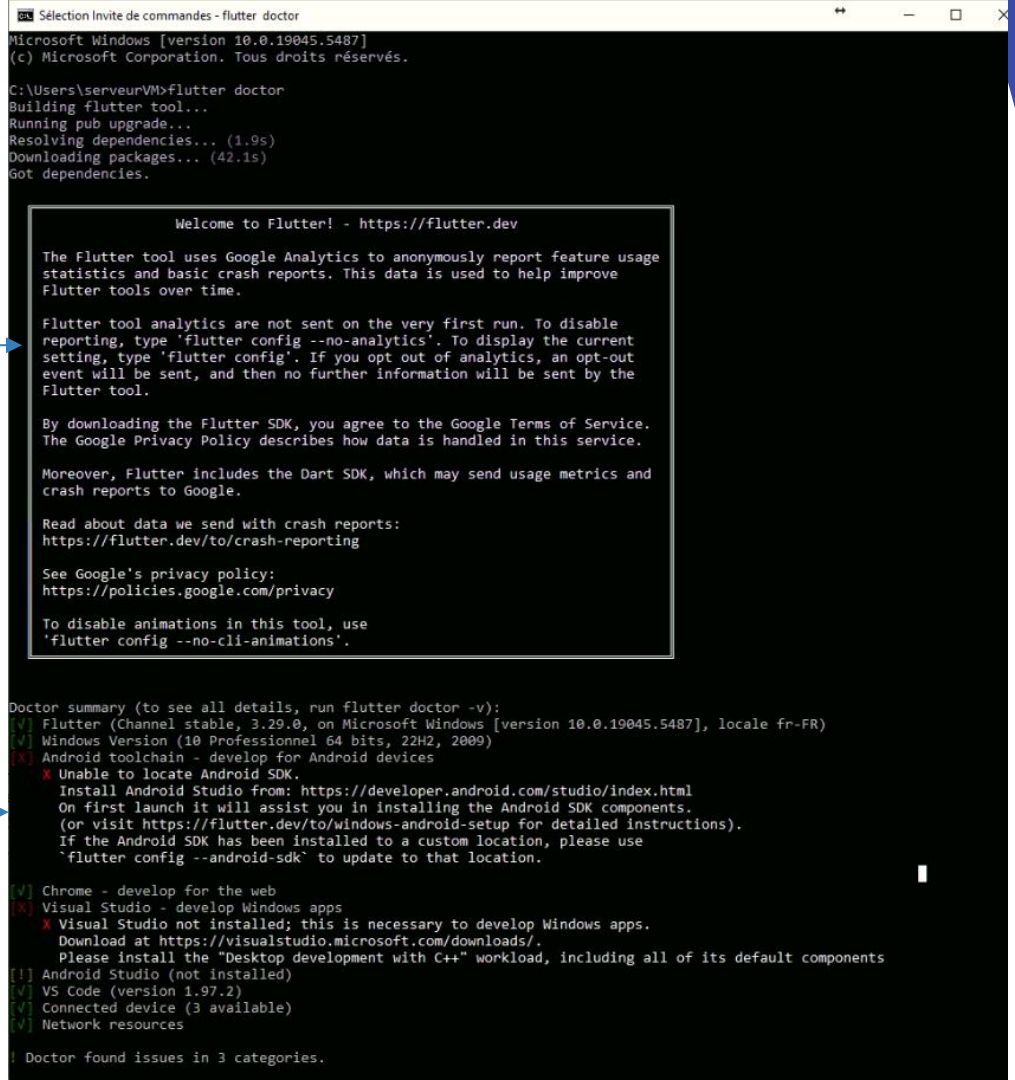
Lancer une invite de commande cmd

flutter doctor

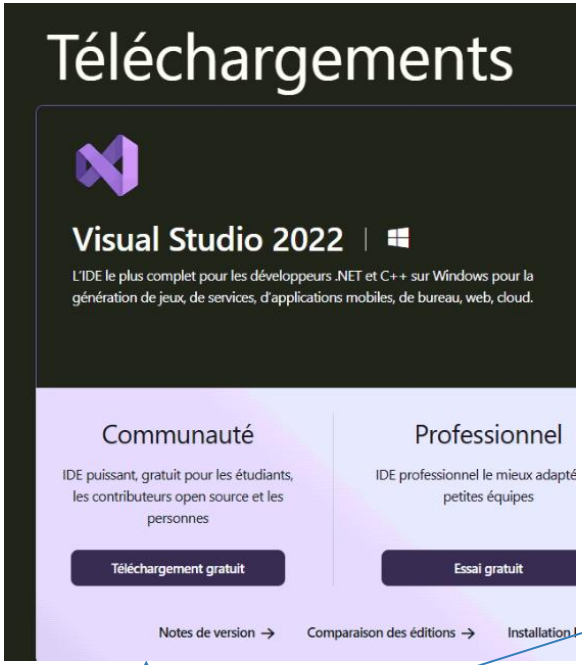
Cela va télécharger ce qu'il manque

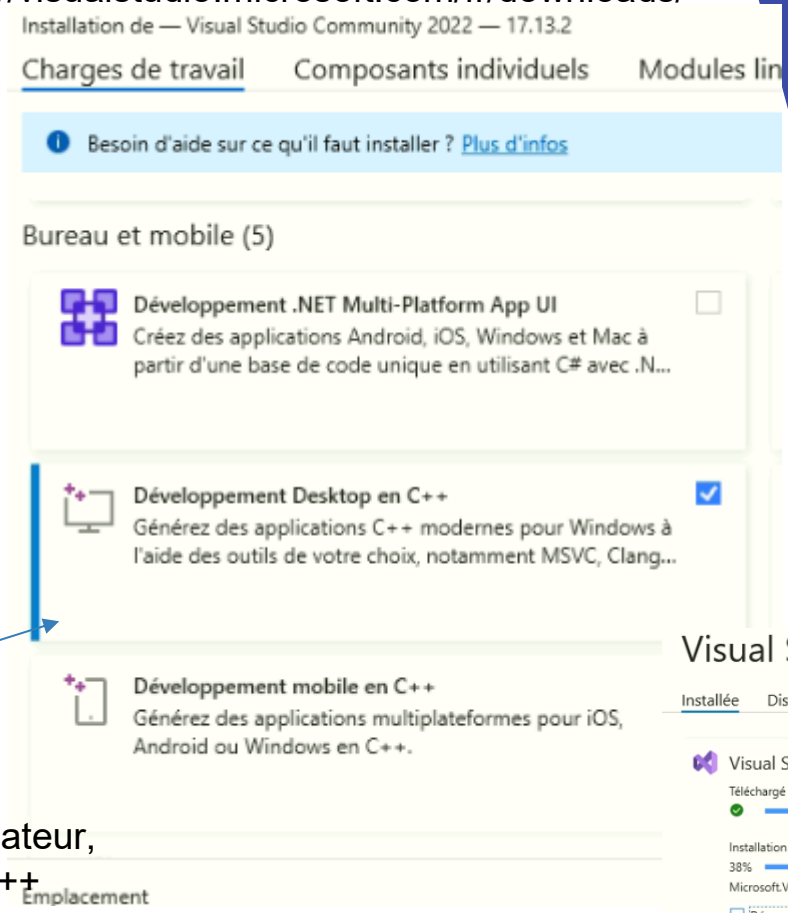
S'il y a l'erreur du SDK android, ce n'est pas grave, on ne va pas faire d'application android

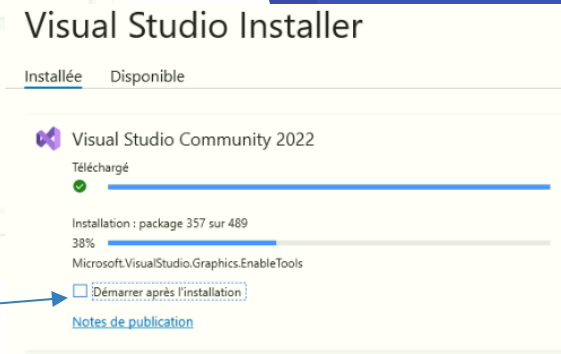
Si erreur Visual Studio, voir suite



Si erreur visual studio : aller sur https://visualstudio.microsoft.com/fr/downloads/







Téléchargement gratuit, lancer l'installateur,
Choisir Développement Desktop en C++
Cliquer sur Installer
Décocher « Lancer après l'installation »

Cette installation peut prendre un peu de temps
Une fois fini, relancer flutter doctor

```
C:\Users\serveurVM>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[√] Flutter (Channel stable, 3.29.0, on Microsoft Windows [version 10.0.19045.5487], locale fr-FR)
[√] Windows Version (10 Professionnel 64 bits, 22H2, 2009)
[X] Android toolchain - develop for Android devices
    X Unable to locate Android SDK.
      Install Android Studio from: https://developer.android.com/studio/index.html
      On first launch it will assist you in installing the Android SDK components.
      (or visit https://flutter.dev/to/windows-android-setup for detailed instructions).
      If the Android SDK has been installed to a custom location, please use
      `flutter config --android-sdk` to update to that location.

[√] Chrome - develop for the web
[√] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.13.2)
[!] Android Studio (not installed)
[√] VS Code (version 1.97.2)
[√] Connected device (3 available)
[√] Network resources

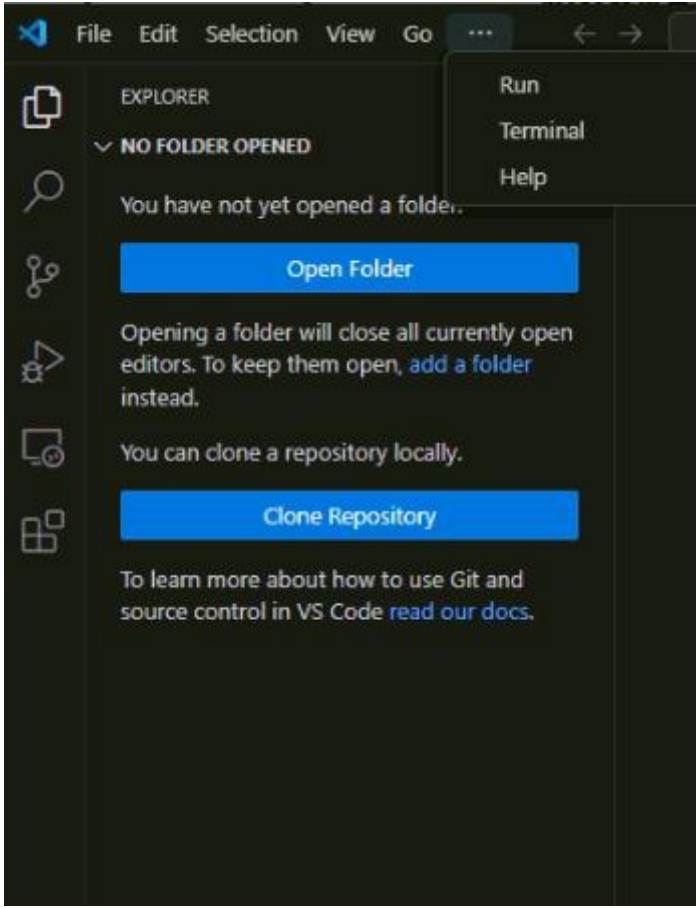
! Doctor found issues in 2 categories.
```


Il ne reste que l’erreur concernant Android

Lancer VS Code

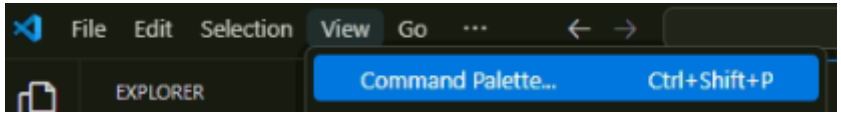
Aller dans extensions

Taper flutter, installer l'extension et accepter l'installation de dart

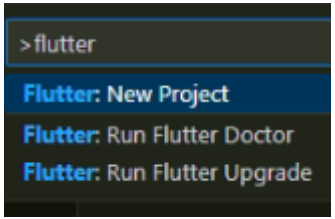


Dans VS Code

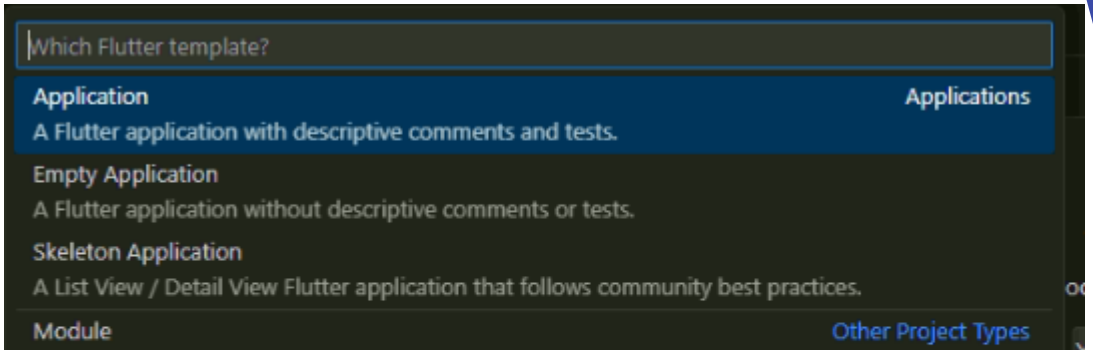
View / Command Palette



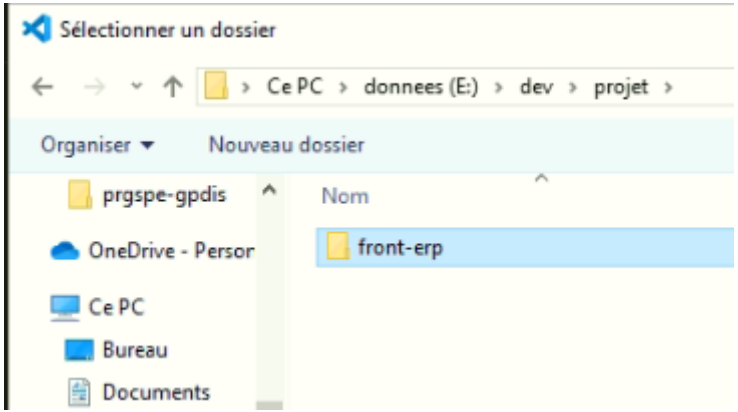
Taper flutter et choisir New Project



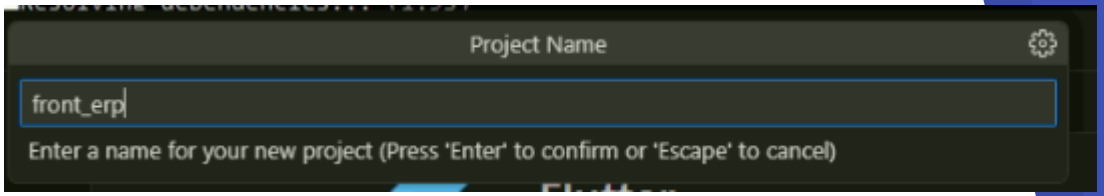
Choisir Application



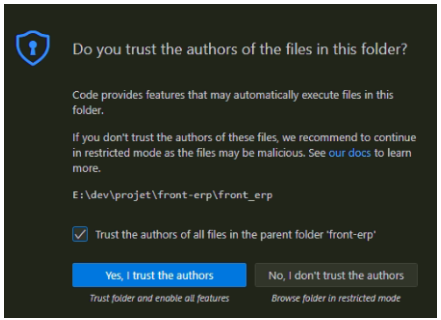
Créer et choisir le dossier de votre choix.
Ex : j'ai créé le dossier front-erp



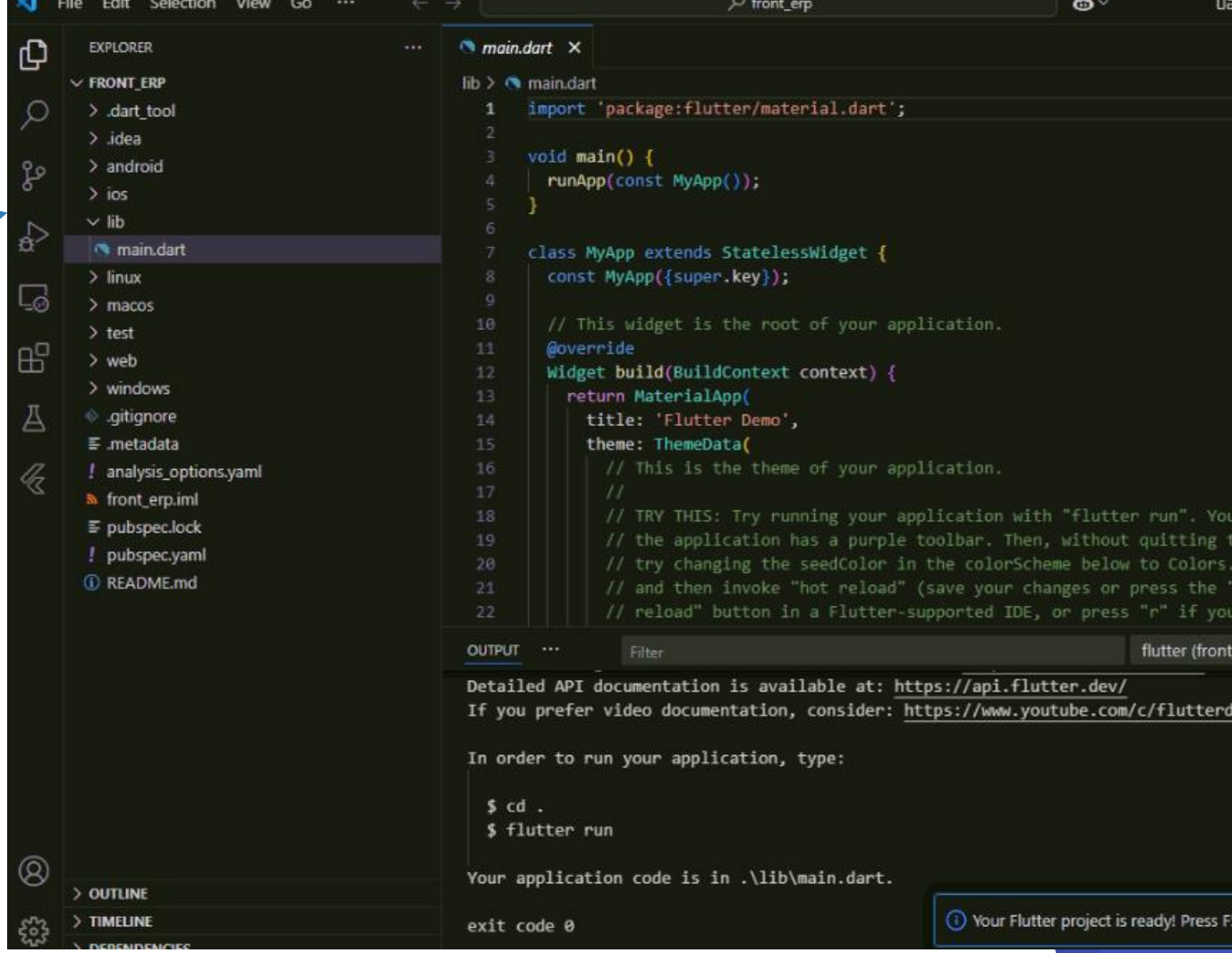
Donner le nom du projet.
Ex : front_erp



Accepter

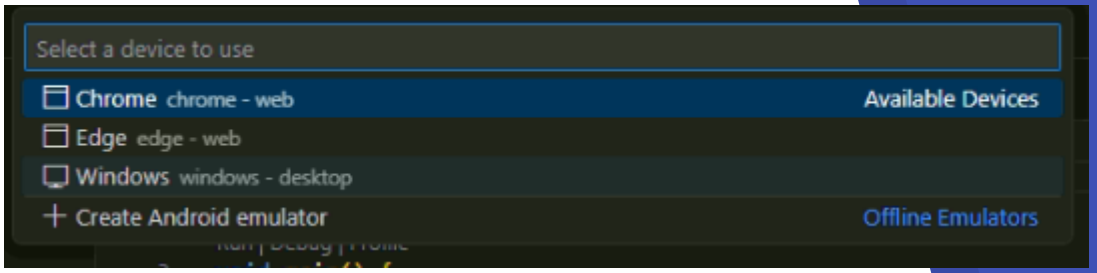


Le projet se créé , cela met quelques minutes, et à la fin, vous obtenez

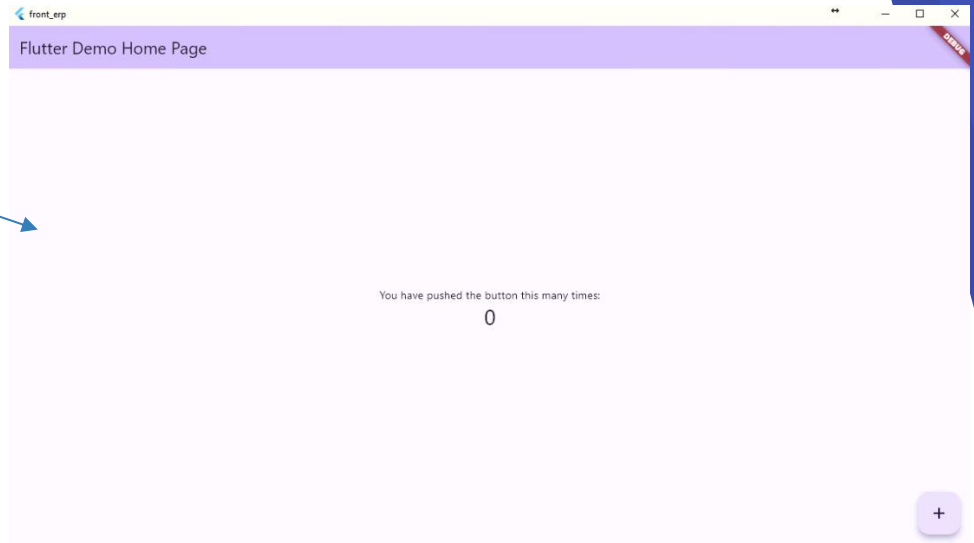


On va tester le projet créé

Faites F5, choisir Windows



Après compilation, l'application de démo se lance

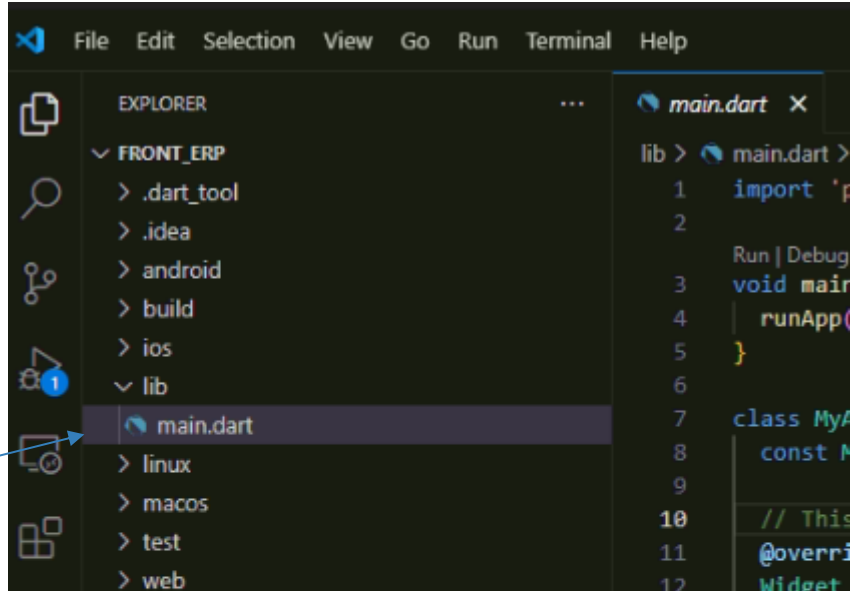


Explications

- Un projet d'application Flutter a été crée

- Le code de l'application se trouve dans le fichier main.dart

Cliquer dessus pour accéder au code source



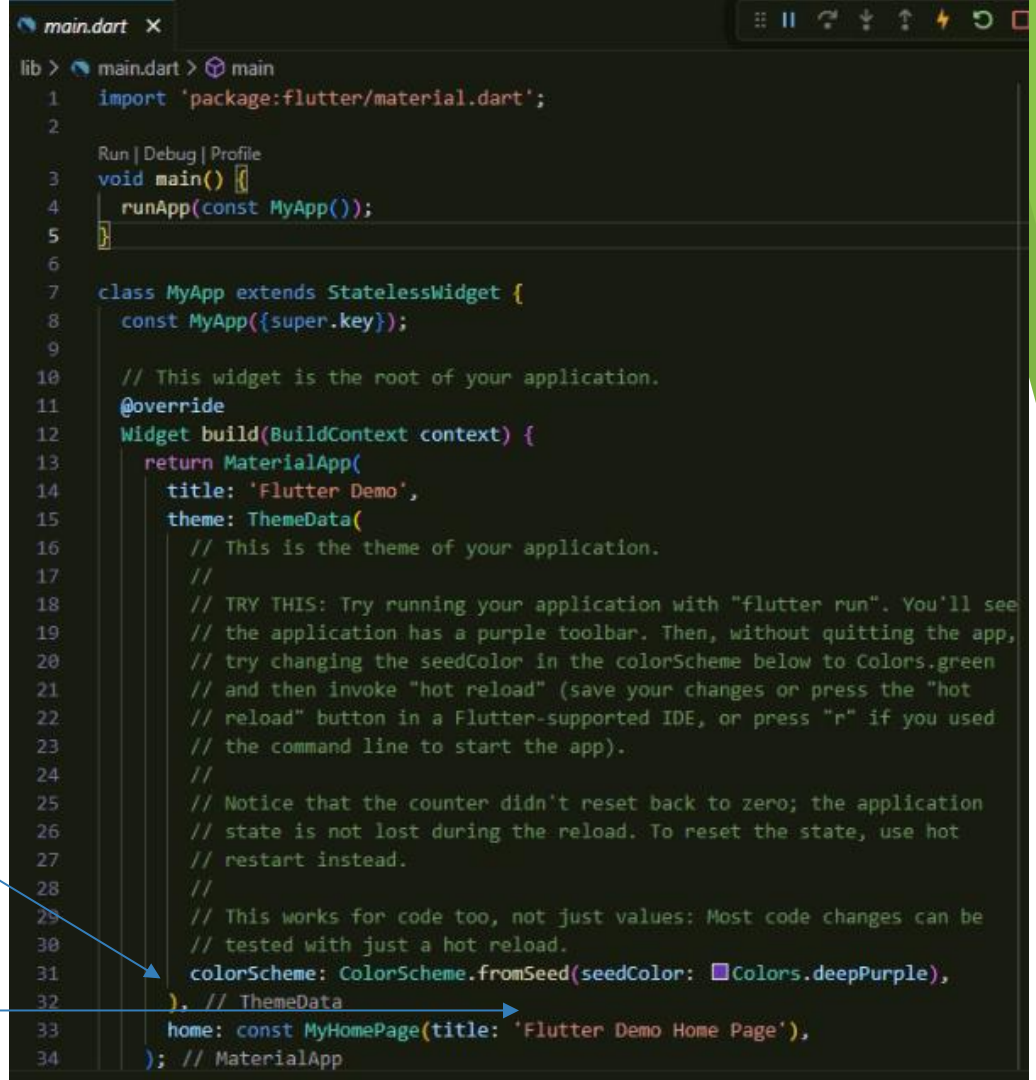
Ici c'est la fonction main, qui crée et lance MyApp

MyApp est un Wigdet, la fonction build permet de définir ce qui va être construit

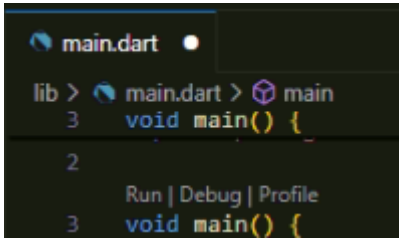
Modifier le titre « Flutter demo » par « Boutique IUT »

Modifier la couleur en retirant .deepPurple, puis remettre le . à droite de Colors, cela ouvre le choix.

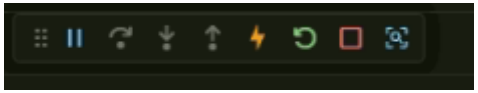
Modifier le titre « Flutter demo » par « Boutique IUT »



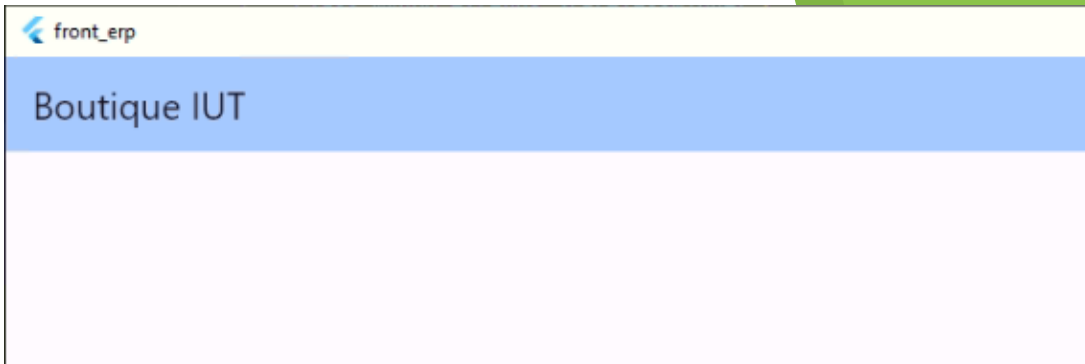
Sauvegarder votre fichier main.dart (ctrl + s)



Utiliser l'icone « éclair »
Pour faire du Hot Reload
C'est-à-dire votre application se recrée avec vos modifications, sans la quitter.



Cela a bien été modifié



Descendez dans le code du main.dart
On arrive au widget MyHomePage

Ici c'est la définition de son State

main.dart

lib > main.dart > MyHomePage

```dart
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  // This widget is the home page of your application. It is stateful, meaning
  // that it has a State object (defined below) that contains fields that affect
  // how it looks.

  // This class is the configuration for the state. It holds the values (in this
  // case the title) provided by the parent (in this case the App widget) and
  // used by the build method of the State. Fields in a Widget subclass are
  // always marked "final".

  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(fn: () {
      // This call to setState tells the Flutter framework that something has
      // changed in this State, which causes it to rerun the build method below
      // so that the display can reflect the updated values. If we changed
      // _counter without calling setState(), then the build method would not be
      // called again, and so nothing would appear to happen.
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
```

Et maintenant ?

Le but est d’afficher les articles avec le json retourné par l’API :

http://polyedre.eu:8000/polyfx/cgi/getart.cgi?frs=401000394